

第2课 实现软件架构

北京交通大学软件学院

目录



01. 架构决策的内容

02. **ASR**识别

03. **ADD**架构设计

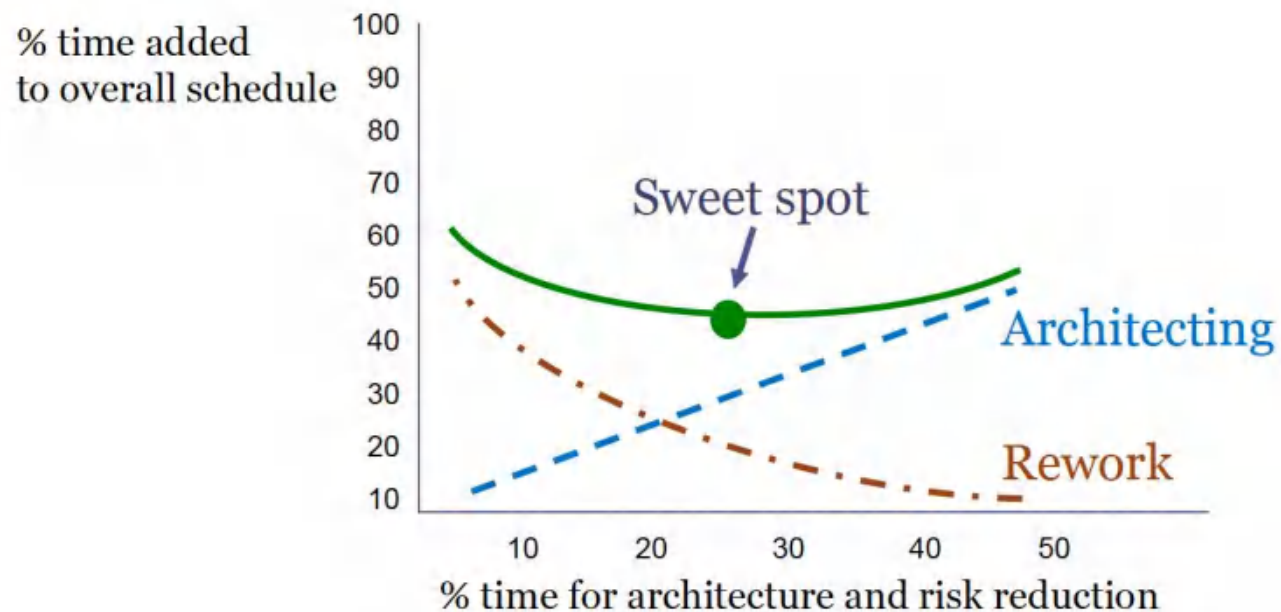
04. 架构评估方法



01 架构决策

多少架构是合适的

- 在过度设计和返工之间作出权衡
- 项目整体时间=
 - +开发时间
 - +架构设计的时间
 - +返工的时间



架构决策包含哪些决定

- 软件架构中重要的决策有：
 - 质量属性
 - 系统结构
 - 依赖关系
 - 对外提供的接口
 - 技术方案

架构决策的反模式 (bad)

- 分析瘫痪
 - 决策无法做出
 - 害怕做出决策
 - 建议： 与团队一起评估决策（并预期会有变更）
 - 等到最后一个负责任的时刻（但不要再拖延）
- 无限讨论（土拨鼠日）
 - 人们不知道为什么做出某个决策
 - 决策被一遍又一遍地讨论
 - 建议： 始终添加适当的理由说明
- 基于邮件的决策
 - 人们丢失、忘记甚至不知道某个决策
 - 建议： 使用架构决策记录 (ADR)

架构要解决的议题

- 风险
- 未知因素
- 问题
- 技术债务
- 理解上的差距
- 腐化
- 环境漂移

风险

- 风险 = 某种可能发生但尚未发生的坏事
- 风险应该被识别和记录。
 - 风险可能作为质量保证 (QA) 场景的一部分出现。
- 风险可以被缓解或接受。
 - 如果可能，应识别缓解任务。
- **Risk = 失败的概率 X 失败的影响**

未知因素

- 有时我们没有足够的信息来判断一个架构是否满足需求。
- 这可能是由于：
 - 需求不明确
 - 隐含的假设
 - 不断变化的需求
 - ...
- 架构评估可以帮助将“未知的未知”转化为“已知的未知”。

问题

- 问题是已经发生的糟糕的事情。
 - 它们产生于设计决策未能按预期奏效之时。
 - 问题也可能因环境变化而出现——
 - 曾经是好主意的决策，现在不再合理。
- 问题可以被修复或接受。
 - 未被修复的问题可能导致技术债务。

技术债

- 技术债务是指在知情或不知情的情况下，做出错误或非最优设计决策而累积的债务。
 - 如果按期偿还（技术债务），债务就会被消除，不会产生进一步的问题。
 - 否则，就会产生利息形式的惩罚。
 - 如果长期不偿还账单，总债务会变得非常庞大，以至于必须宣告破产。
 - 在软件领域，这可能意味着产品被放弃。
- 技术债务有几种类型：
 - 代码债务：不良或不一致的编码风格。
 - 设计债务：设计缺陷。
 - 测试债务：缺乏测试、测试覆盖率不足等。
 - 文档债务：过时的文档，或重要事项缺乏文档。

架构腐化

- 设计架构与实际构建架构之间的差距
 - 实际实现的系统几乎不可能完全按照架构师最初设想的方式呈现。
 - 如果缺乏警惕，架构会每天偏离计划的设计一点点，直到最终实现的系统与最初的计划几乎没有相似之处。
- 架构上显而易见的代码可以缓解这种偏差。

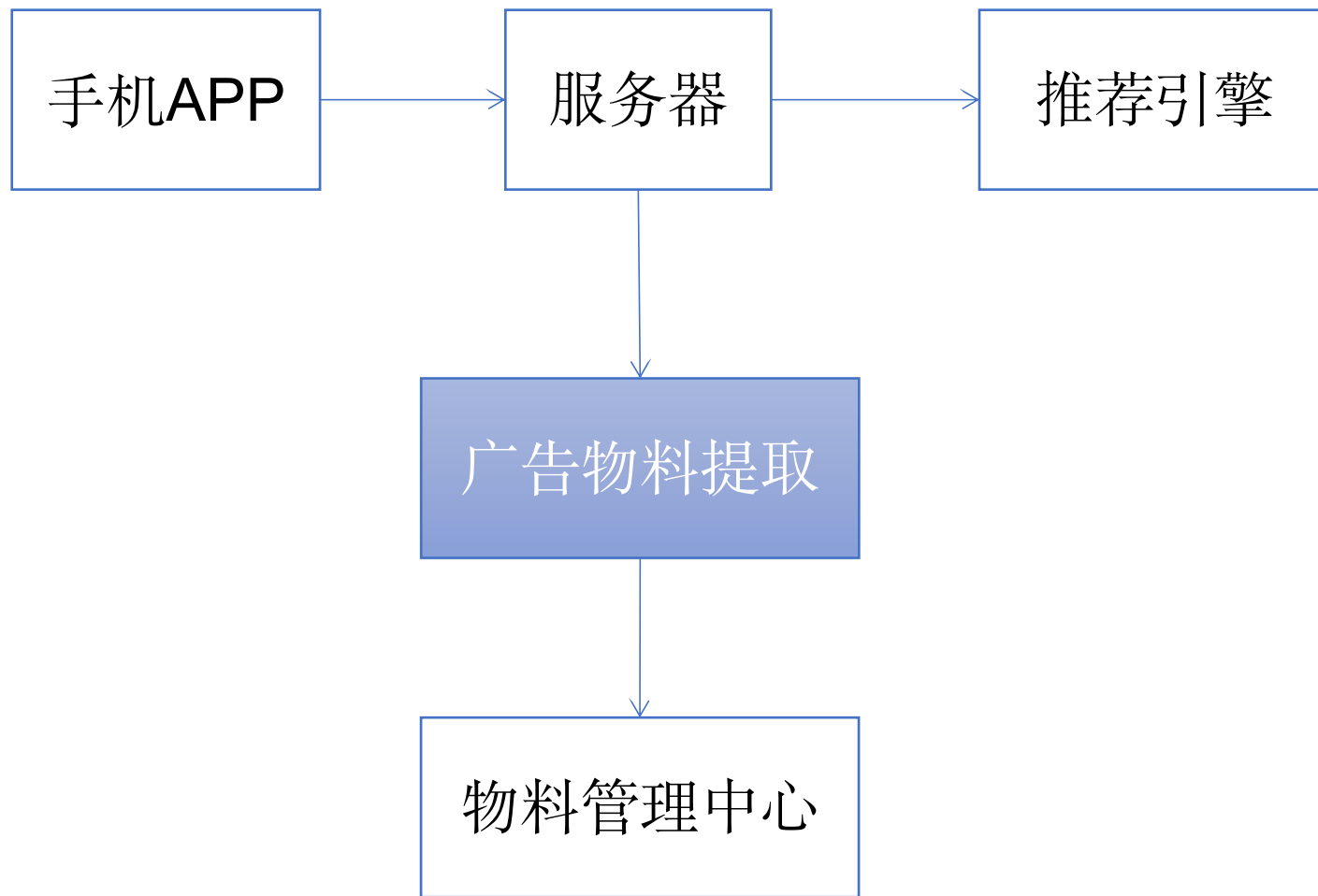
环境漂移

- 背景驱动因素变化
 - 这种情况发生在业务或背景驱动因素在做出设计决策后发生变化时。
 - 因此，有必要不断地重新审视需求，并采用进化架构。



02 识别ASR

需求：广告物料提取



提供一个高可用、高性能的广告物料加载服务，确保将最新的广告物料及时、准确地送达渲染服务器。

满足千万级广告物料的存储和检索需求，并具备应对每日百万级新增物料的能力。广告物料加载延迟控制在 **1ms** 以内。

降低因物料加载失败导致的广告展现损失。

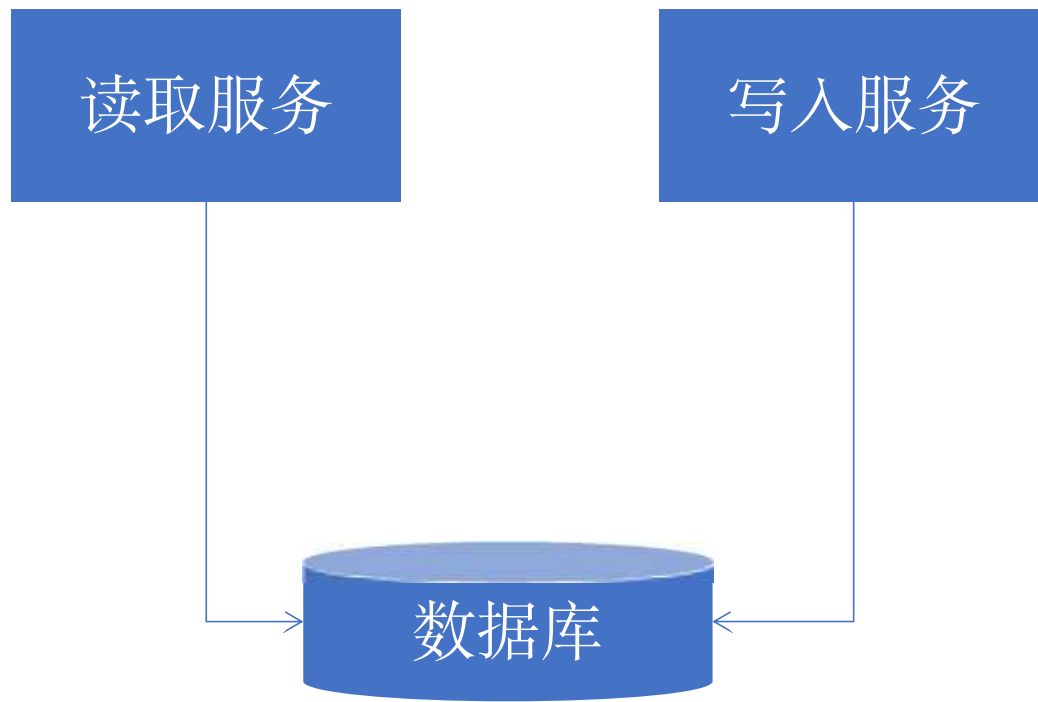
广告物料是商业机密，禁止泄露。

什么是ASR

所谓架构攸关需求 (Architecturally Significant Requirement, ASR), 并非泛泛之需, 而是对系统架构产生深远影响的关键需求。换言之, 若缺失此等需求, 整个架构很可能发生颠覆性的变化, 呈现截然不同的面貌。

1. ASR不是需求: 功能性需求 VS 非功能性需求
2. User Story是需求的一种形式:
 1. 作为一个 [用户角色], 我想要 [目标], 以便于 [理由/价值]
3. 一个User Story包含多个ASR

广告物料提取的架构设计 (无ASR)



如何识别ASR

需求文档

访谈项目干系人

理解业务目标

使用效用树

通过需求文档识别ASR-理论

1. 职责分配 (Allocation of Responsibilities):
 - 职责的规划演进, 用户角色, 系统模式, 主要处理步骤, 商业软件包。
2. 协调模型 (Coordination Model):
 - 协调的属性 (及时性、现时性、完整性、正确性和一致性), 外部元素名称, 协议, 传感器或执行器 (设备), 中间件, 网络配置 (包括其安全属性)。
3. 数据模型 (Data Model):
 - 处理步骤, 信息流, 主要领域实体, 访问权限, 持久性, 演进需求。
4. 资源管理 (Management of Resources):
 - 时间, 并发性, 内存, 调度, 设备, 能源消耗, 软资源 (缓冲区、队列等)。
5. 架构元素映射 (Mapping among Architectural Elements):
 - 团队协作计划, 处理器, 处理器系列, 处理器演进, 网络配置。
6. 绑定时间决策 (Binding Time Decisions):
 - 功能的扩展或灵活性, 区域差异, 语言差异, 可移植性, 校准, 配置。
7. 技术选型 (Choice of Technology): 指定的技术, 技术的变更 (计划内和计划外)。

通过需求文档识别ASR-例子

提供一个高可用、高性能的广告物料加载服务，确保将最新的广告物料及时、准确地送达渲染服务器。

满足千万级广告物料的存储和检索需求，并具备应对每日百万级新增物料的能力。广告物料加载延迟控制在1ms以内。

降低因物料加载失败导致的广告展现损失。

广告物料是商业机密，禁止泄露。

通过访谈识别ASR- (1)

第二步：业务/使命介绍 (Business/Mission Presentation): 奠定基础

- 代表系统背后业务关注点的利益相关者（通常是经理或管理代表）花费大约一个小时介绍系统的业务背景、广泛的功能需求、约束以及已知的质量属性需求。将在后续步骤中完善的质量属性主要来自此步骤中提出的业务/使命需求。

第五步：场景头脑风暴 (Scenario Brainstorming): 激发创意

- 每个利益相关者表达一个代表他或她对系统关注的场景。引导者确保每个场景都有明确的刺激和响应。引导者确保对于每个架构驱动因素，至少存在一个代表性场景。

第七步：场景优先级排序 (Scenario Prioritization): 确定重点

- 翻译：场景的优先级排序是通过为每个利益相关者分配一定数量的选票来完成的，选票数量等于整合后生成的场景总数的 30%。利益相关者可以将任意数量的选票分配给任何场景或场景组合。计算选票，并相应地对场景进行优先级排序。

通过访谈识别ASR- (2)

与广告服务负责人:

- 负责人: 1000 QPS, 1ms 延迟, 压力很大。
- 架构师: 请求构成是怎样的?
- 负责人: 根据用户画像等信息检索物料, 数量不多但速度要快, 物料要实时更新。
- 架构师: 除了性能, 还有什么关注点?
- 负责人: 稳定性! 每秒钟10,000元。不能影响公司收入。

与服务器管理负责人:

- 负责人: 单机 运行1000 小时 10% 概率重启
- 架构师: 服务器配置和死机原因?
- 负责人: 配置32C256G, 8个机房, 死机原因是不可控因素。
- 架构师: 除了稳定性, 还有什么关注点?
- 负责人: 成本! 单服务器20000元, 尽量降低服务器成本。

通过业务目标识别ASR (1)

1. **目标** (Goal): 任何业务目标。
2. **目标来源** (Goal-source): 出处。
3. **目标主体** (Goal-subject): 拥有该目标并希望它实现的利益相关者。
4. **目标对象** (Goal-object): 目标适用的实体。
5. **环境** (Environment): 此目标的背景。
6. **目标衡量** (Goal-measure): 一个可测试的测量方法，用于确定如何知道目标是否已实现。目标衡量通常应包括一个时间组成部分，说明应实现目标的时间。
7. **谱系和价值** (Pedigree and value): 目标的谱系从更高的视角说明了目标的价值。

对于正在开发的在线购物平台，**公司管理层和股东**希望**公司的利润和客户满意度**在**竞争激烈的电商市场、快速变化的技术环境以及用户对个性化体验的需求日益增长的背景下**，**在一年内**将平台的**用户数量**增加**50%**，并将**客户满意度评分**提高到**4.5分**（满分5分），这样公司的整体利润就能保持**10%**的增长速度，并且为后续**5年**的持续增长提供动力。

TABLE 16.4 General Scenario Generation Table for Business Goals

2. Goal-subject	3. Goal-object	5. Goal	4. Environment	6. Goal-measure (examples, based on goal categories)	7. Value
Any stakeholder or stakeholder group identified as having a <i>legitimate</i> interest in the system	Individual System Portfolio Organization's employees Organization's shareholders Organization Nation Society	Contributing to the growth and continuity of the organization Meeting financial objectives Meeting personal objectives Meeting responsibility to employees Meeting responsibility to society Meeting responsibility to state Meeting responsibility to shareholders Managing market position Improving business processes Managing quality and reputation of products Managing change in environmental factors	Social (includes political) Legal Competitive Customer Technological	Time that business remains viable Financial performance vs. objectives Promotion or raise achieved in period Employee satisfaction; turnover rate Contribution to trade deficit/surplus Stock price, dividends Market share Time to carry out a business process Quality measures of products Technology-related problems Time window for achievement	1– <i>n</i> 1–10 H-M-L Resources willing to expend

通过理解业务目标识别ASR (2)

1. 贡献于组织的增长和持续性
2. 达成财务目标: 公司目标, 个人目标, 经理目标
3. 达成个人目标: 学习新技术, 提升个人声望
4. 履行对员工的责任: 人身安全, 工作负荷, 个人职业规划
5. 履行对社会的责任: 社会伦理道德, 缓解社会矛盾
6. 履行对国家的责任: 进出口限制, 支持政府活动, 管制措施
7. 履行对股东的责任: 与财务目标类似, 但是部分股东有特殊的要求
8. 管理市场地位: 增加市场份额, 申请专利保护, 寻求竞争优势
9. 改进业务流程: 开发新市场, 开发新产品, 提高服务质量
10. 管理产品声誉: 品牌营销, 发掘潜在用户
11. 管理环境因素的变化: 老龄化, 用户需求变化

PALM: 从业务目标中识别ASR (3)

1. PALM 概述: 介绍 PALM、它解决的问题、步骤以及预期结果。
2. 业务驱动因素: 客户组织的目标是什么? 开发组织的目标是什么?
3. 架构驱动因素: 架构师简要介绍驱动业务和质量属性的需求
4. 分析业务目标: 使用标准的业务目标类别来引导讨论,
 1. 我们捕获一组重要业务目标。
 2. 业务目标被详细阐述并表达为场景。
 3. 合并相似业务目标——消除重复。
 4. 然后, 参与者进行优先级排序, 以确定最重要的目标。
5. 识别ASR。 针对每个业务目标场景, 明确所需的ASR。
6. ASR关联业务目标。 我们确定ASR在支持哪些业务目标。 例如: 为什么有 40 毫秒的性能要求? 为什么不是 60 毫秒? 还是 80 毫秒?
7. 练习总结。 回顾结果、后续步骤和参与者反馈。

通过理解业务目标识别ASR (4) -例子

提供一个高可用、高性能的广告物料加载服务，确保将最新的广告物料及时、准确地送达渲染服务器。

满足千万级广告物料的存储和检索需求，并具备应对每日百万级新增物料的能力。广告物料加载延迟控制在1ms以内。

降低因物料加载失败导致的广告展现损失。

广告物料是商业机密，禁止泄露。

通过业务目标的识别，发现本次业务目标是：

1. 改进业务流程，提高产品质量
2. 达成公司的财务目标

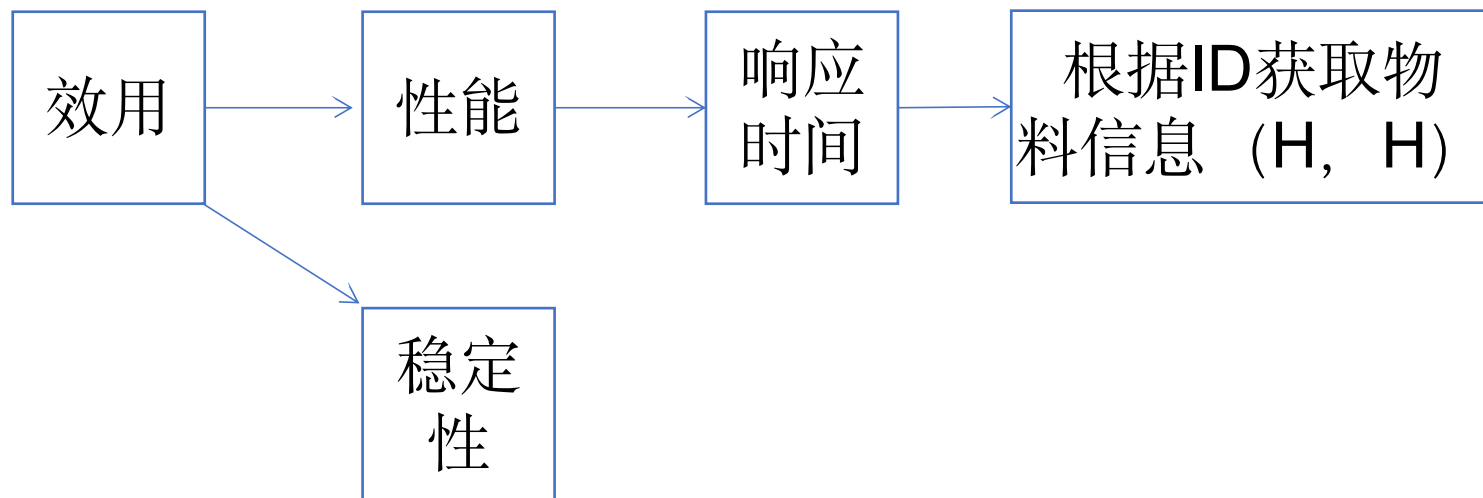


发现当前的物料管理服务存在以下问题：

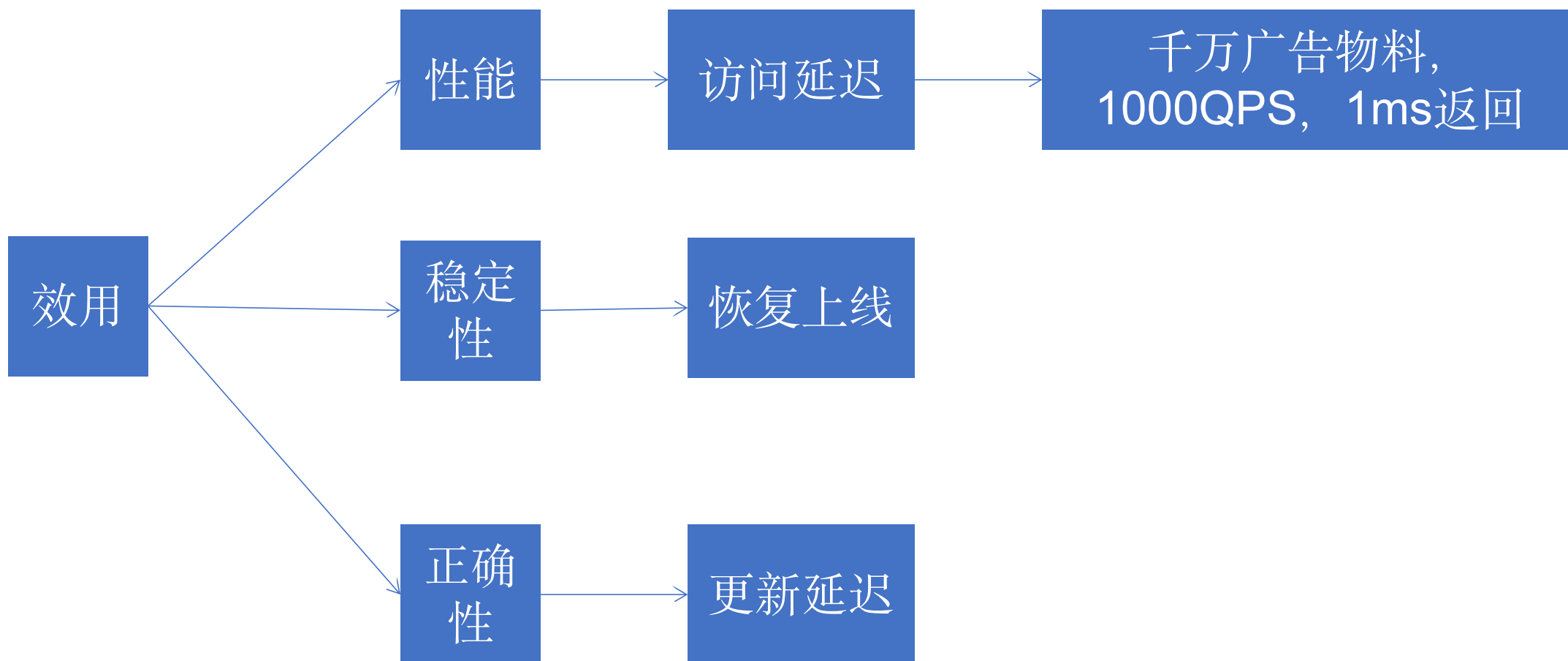
1. 稳定性问题：每周down机 5分钟；
2. 容量问题：旧的物料不能删除，已达上限；
3. 性能问题：QPS 超过800时，耗时超过10

通过效用树识别ASR- (1)

1. 根节点
2. 质量属性
3. 指标分解
4. 关联ASR
5. 业务价值 (HML)
6. 架构影响 (HML)
7. 复查



通过效用树识别ASR- (2)



各种ASR识别方法组合使用

1. 首先使用需求文档
2. 如果需求文档写得不够清晰，使用PALM
3. 如果你发现关键的项目干系人没有提供反馈，进行interview
4. 构建一个效用树进行分析

ASR识别结果

一级属性	指标分解	需求场景描述	优先级 (架构, 业务)
容量	当前容量	当前容量xxx条	H, L
	增加和删除	每日新增xx条, 删除xx条	M, L
性能	响应时间	最大延迟<10ms 99%分为延迟<3ms 平均延迟<1ms	H, H
	生效时间	广告修改后生效时间<10ms	M, H
稳定性	接口成功率	所有调用的成功率99.99%	H, H
	服务器恢复延迟	当一个服务器出现故障, 下线, 到新服务器开始提供服务的时间, 小于100s	H, H
	服务器故障检测延迟	当服务器出现故障, 到系统发现该服务器出现故障, 时间小于10s	M, H



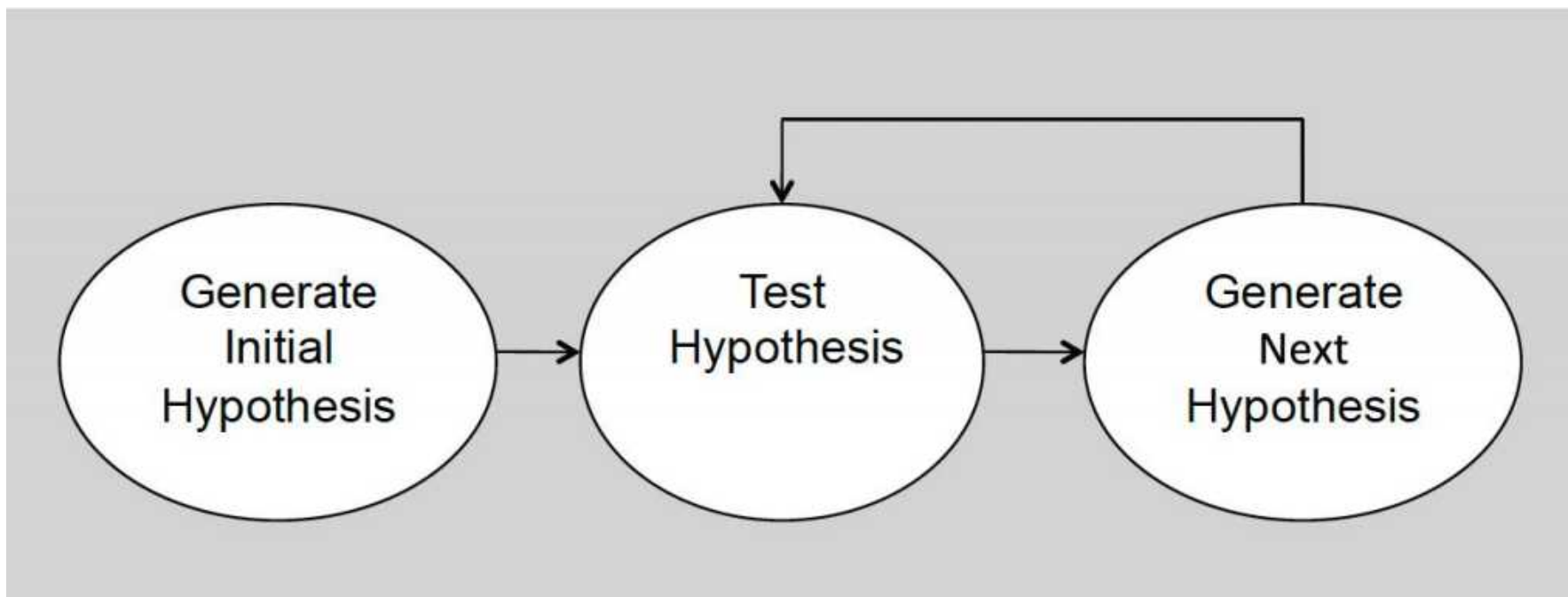
03

ADD: 属性驱动的设计

什么是ADD方法

属性驱动设计（Attribute-Driven Design, ADD）方法是一种生成与测试：

1. 它将必须满足的**需求数量控制**在一个人类可实现的范围内。
2. ADD输出主要设计方法，不是每个细节上都完整的架构
3. 这个架构让其他团队开始工作，同时架构师继续完善设计。



什么是ADD

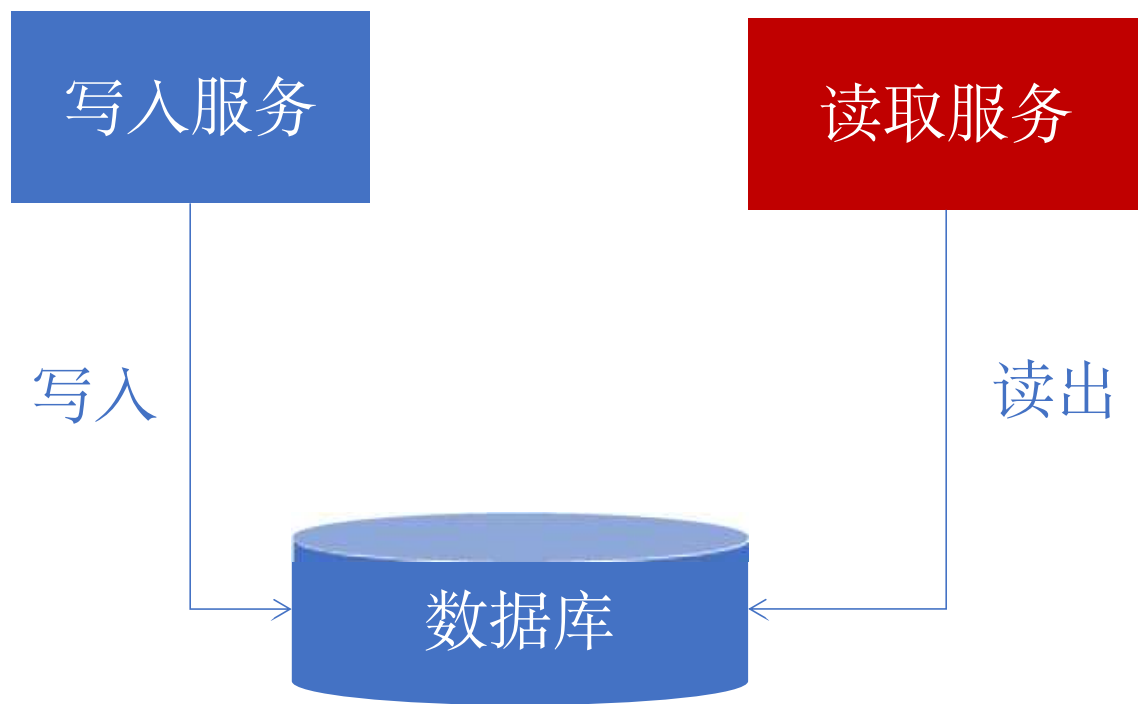
1. 选择要设计的系统元素。
2. 识别所选系统元素的ASR。
3. 为所选元素生成设计方案：使用现有系统、框架、模式和策略。
4. 验证方案是否满足ASR：
5. 重复步骤1-4，直到满足所有ASR，以供实施者使用。

迭代0-Step1 选择元素进行设计-理论

选择的方法:

1. 第一次迭代是定义系统的高层组件，这些组件共同实现系统的全部功能。
2. 第二次迭代将选取一个元素进行设计，从而产生更细粒度的元素。
3. 第三次迭代将选取另一个元素：
 - a. 可以是整个系统的顶层子元素；
 - b. 也可以是子元素中创建的下一代子元素。

迭代0-Step1 选择元素进行设计-例子



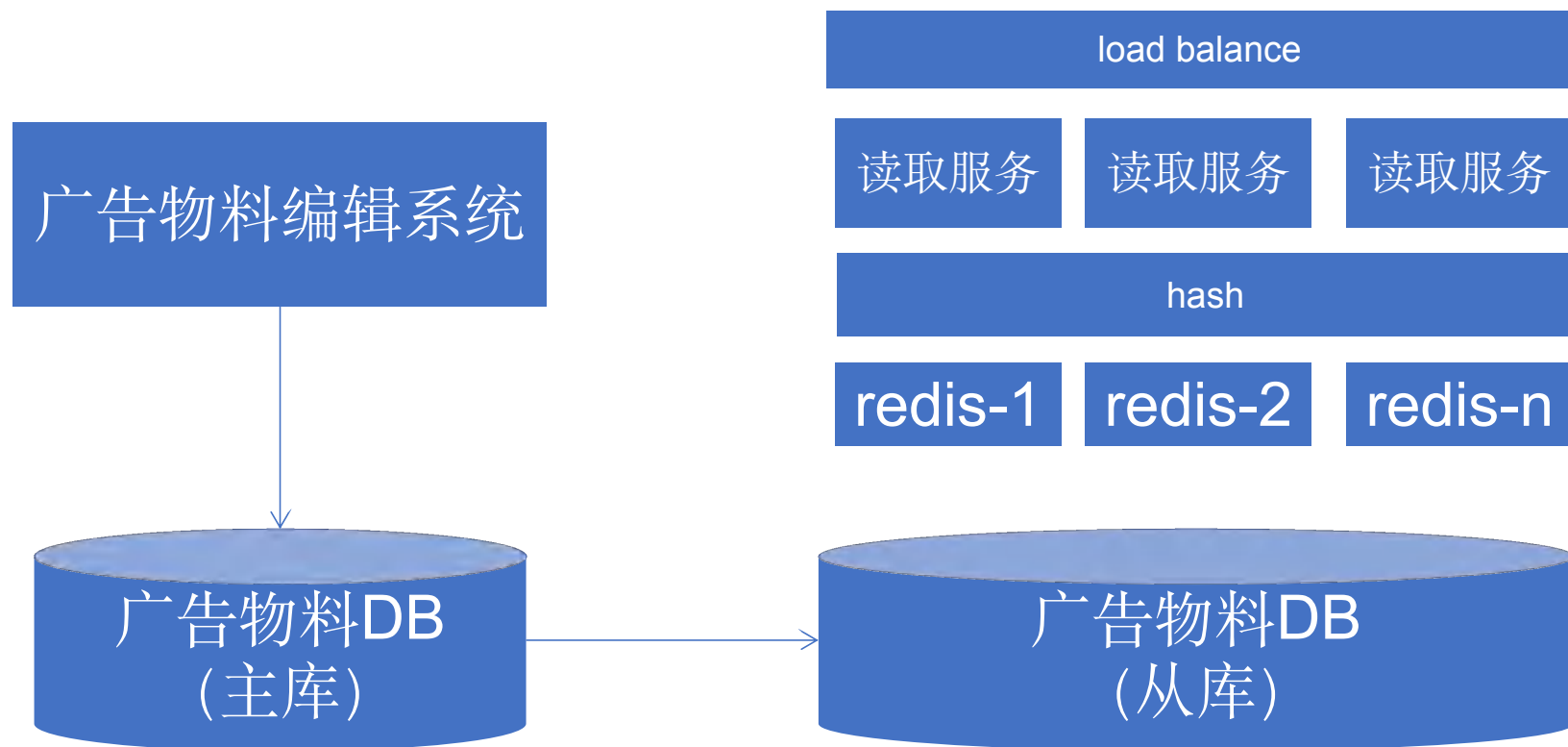
迭代0-Step2选择ASR进行测试

这个方案不能满足性能要求,

1. 数据库的最大并发连接500;
2. 即使能连接DB, 根据测试发现, 使用ID查询数据库的耗时是6ms;



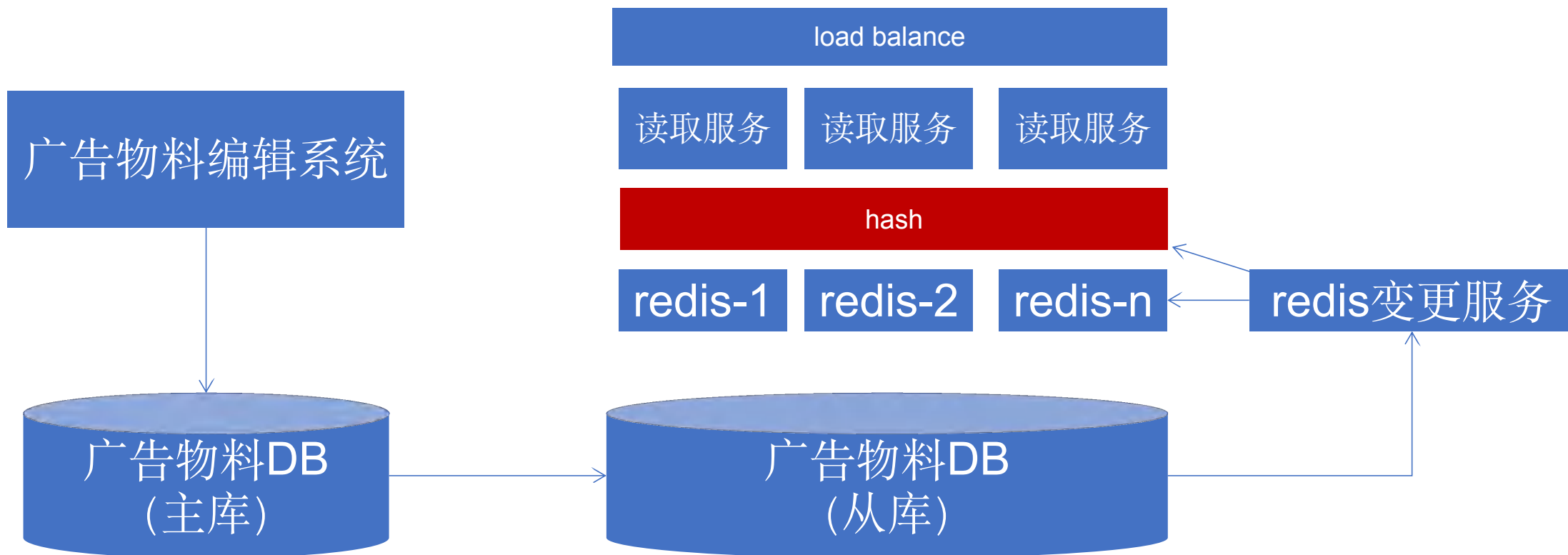
迭代0-产出



1. 单个redis可以支持100万广告;
2. 在QPS为100时, 单次查询耗时500ns;
3. 读取服务无状态;
4. redis如果失败, 可以直接读db;

迭代1

迭代1-Step1 选择元素进行设计



迭代1-Step2 识别该元素的ASR-理论

一级属性	指标分解	需求场景描述	优先级 (架构, 业务)
容量	当前容量	当前容量xxx条	H, L
	增加和删除	每日新增xx条, 删除xx条	M, L
性能	响应时间	最大延迟<10ms 99%分为延迟<3ms 平均延迟<1ms	H, H
	生效时间	广告修改后生效时间<10ms	M, H
稳定性	接口成功率	所有调用的成功率99.99%	H, H
	服务器恢复延迟	当一个服务器出现故障, 下线, 到新服务器开始提供服务的时间, 小于100s	H, H
	服务器故障检测延迟	当服务器出现故障, 到系统发现该服务器出现故障, 时间小于10s	M, H

迭代1-Step3 产生一个设计方案V1

既然需要把5000万的广告物料存储到50台redis服务器上，最简单的方法是随机分配。

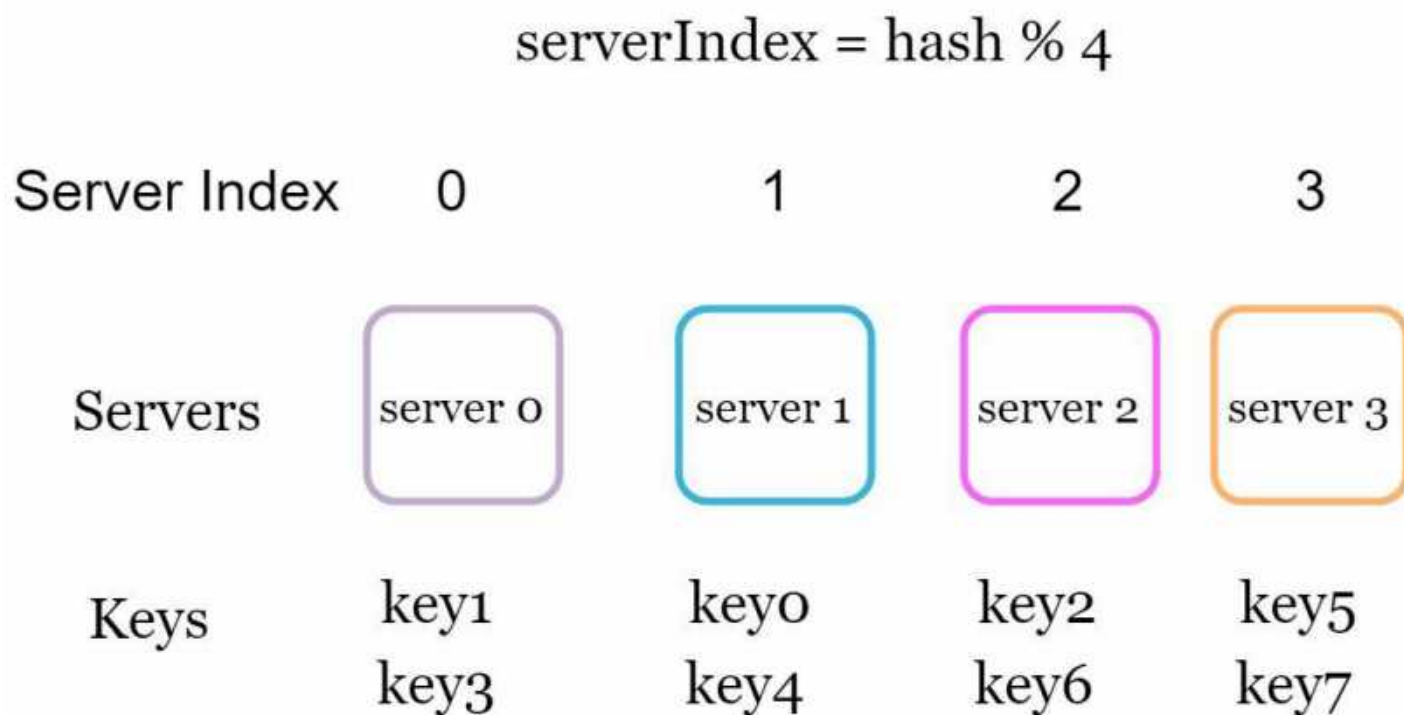
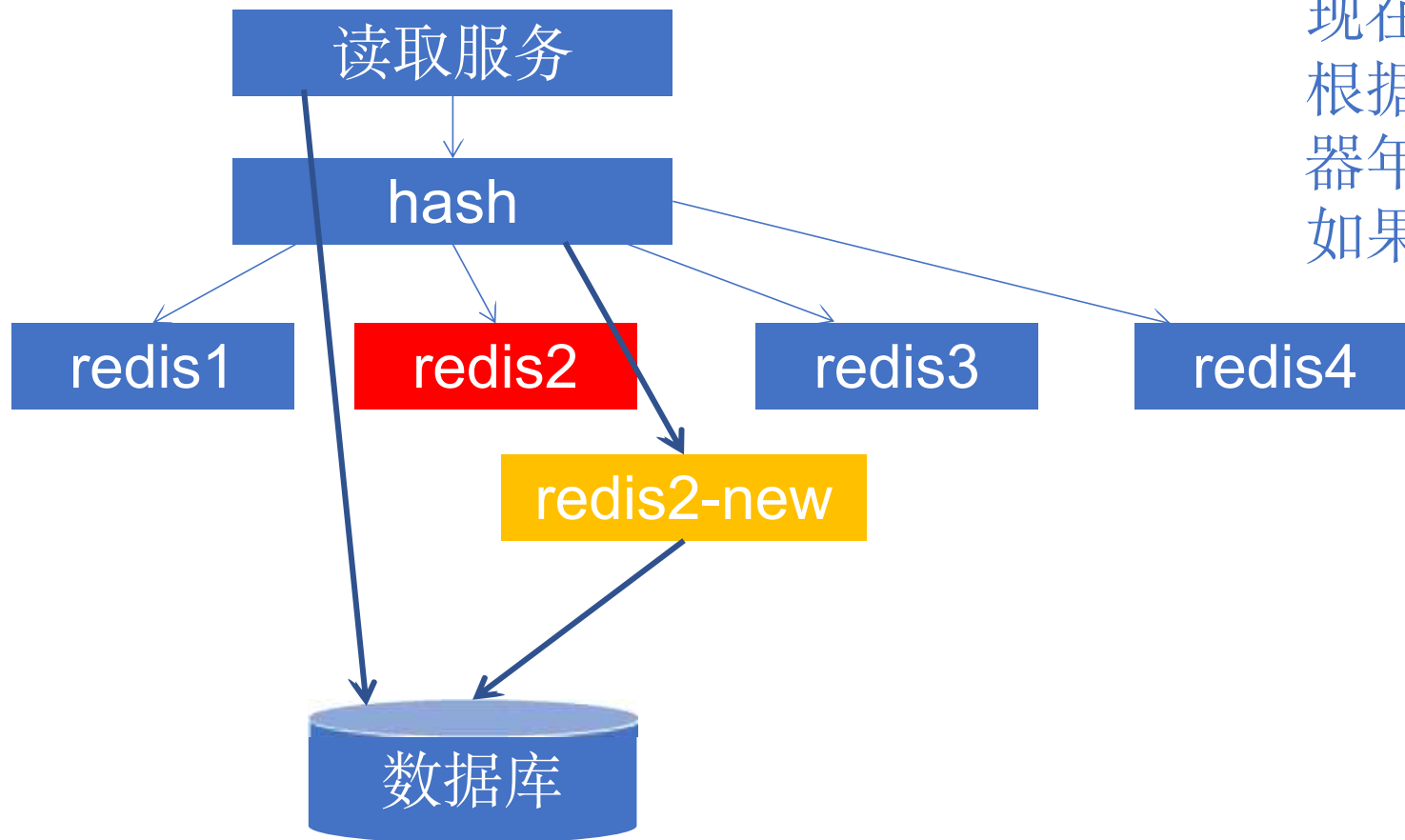


Figure 5-1

迭代1-Step4 选择ASR进行检验 (1)



【性能-响应时间】
现在考虑机器出故障的情况。
根据IT部门的统计，每台Redis服务器年故障率是20%。
如果一台Redis出现故障？怎么处理

迭代1-Step4 选择另一个ASR进行校验 (2)

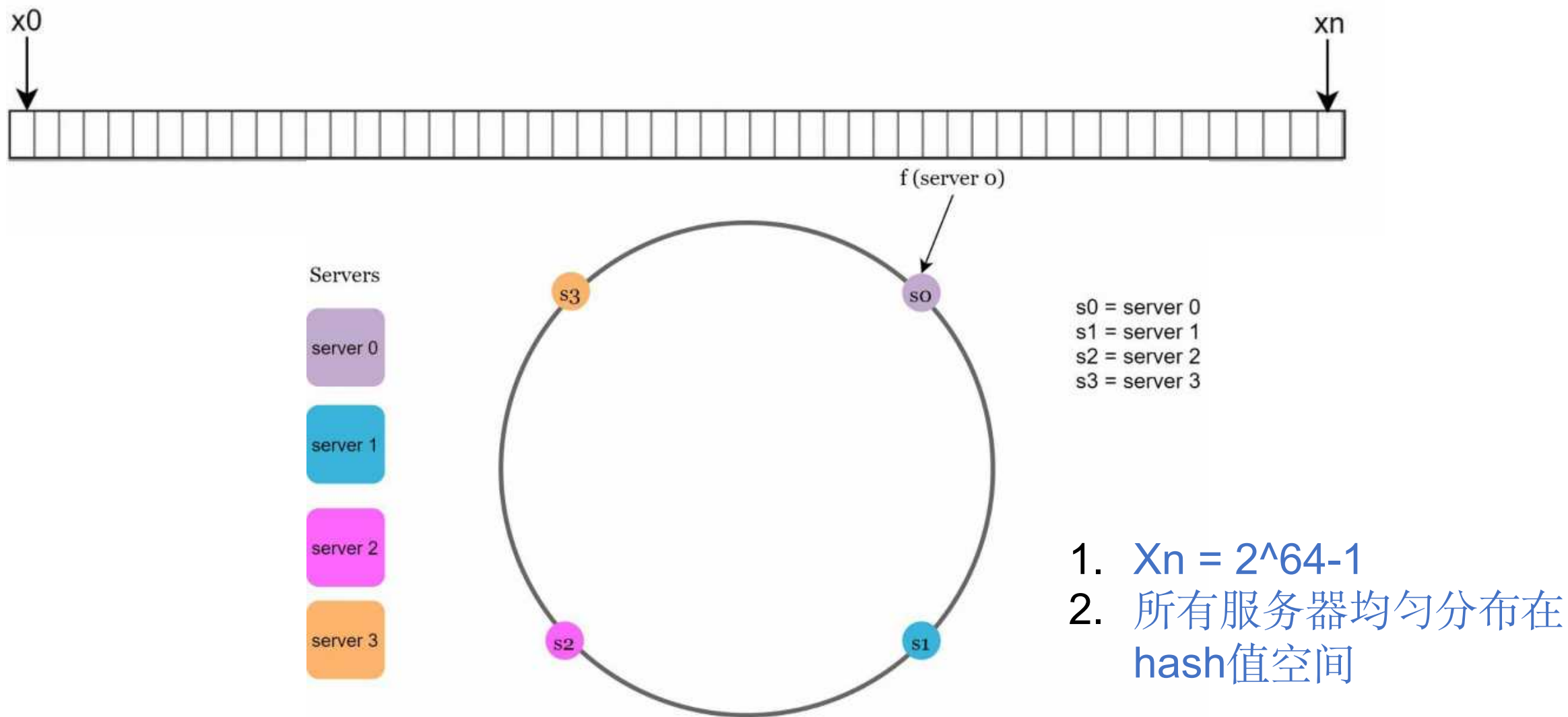
广告物料每日新增100万。如果需要增加redis服务器应该怎么办？



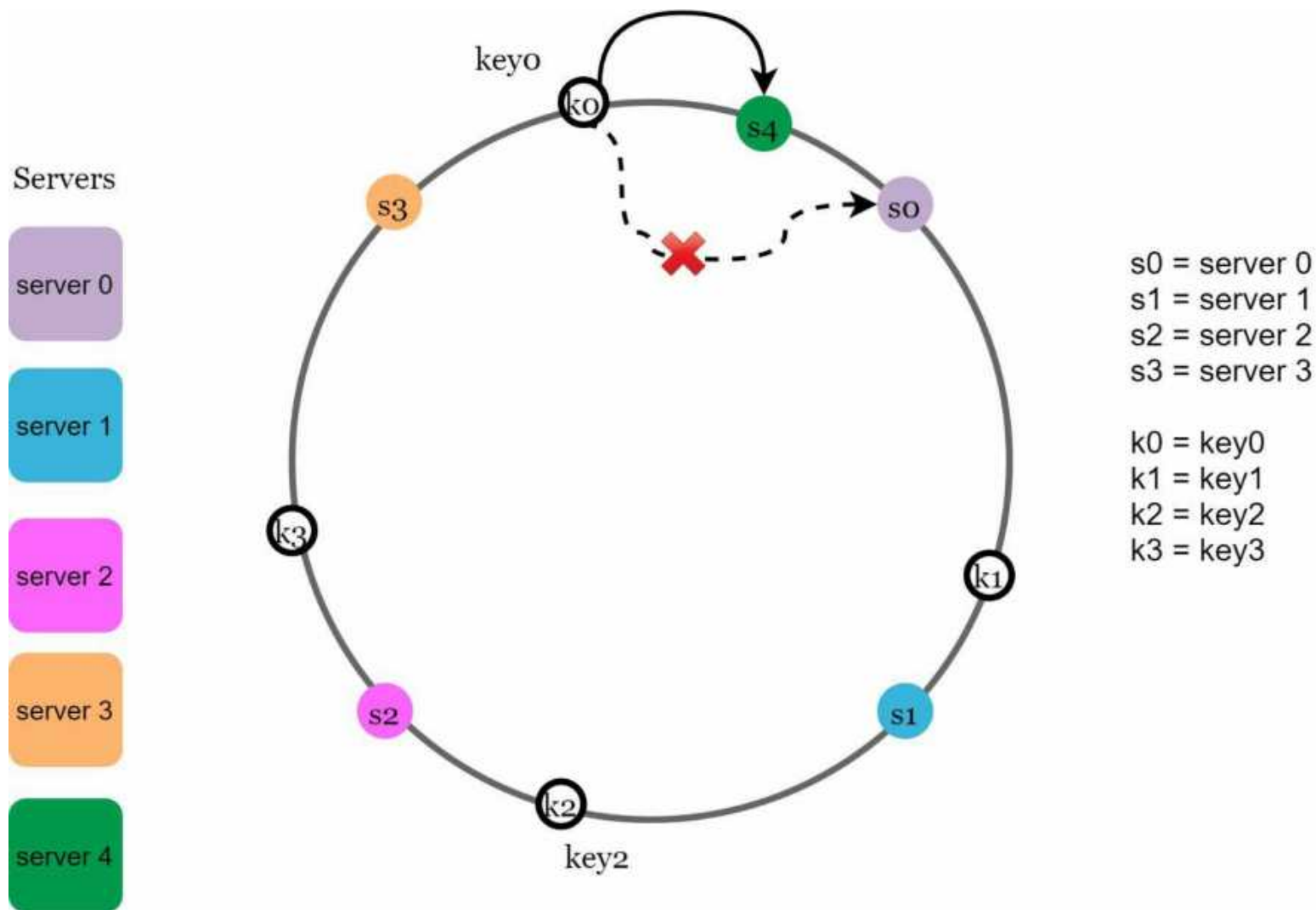
几乎每个redis都需要重新加载key。db将会成为瓶颈。
本次ASR的校验，不通过。



迭代1-Step3 产生设计方案 V2



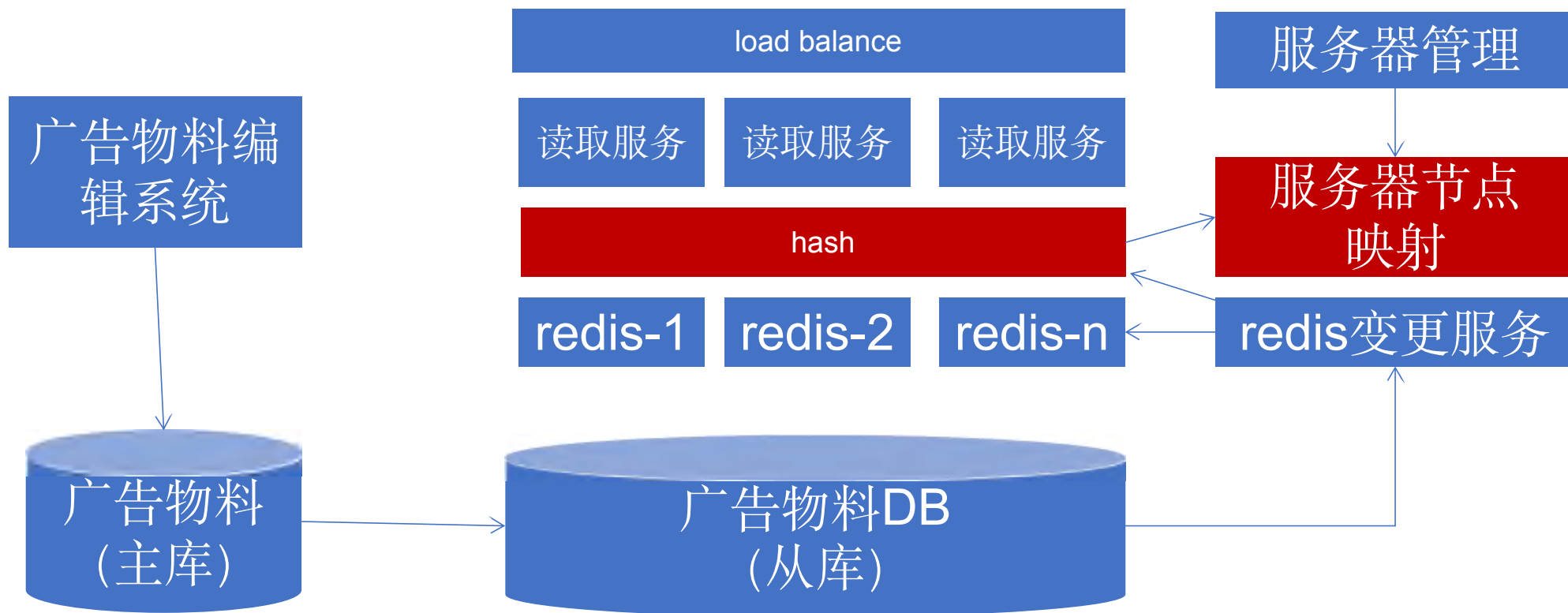
迭代1-Step4选择ASR进行校验 (3)

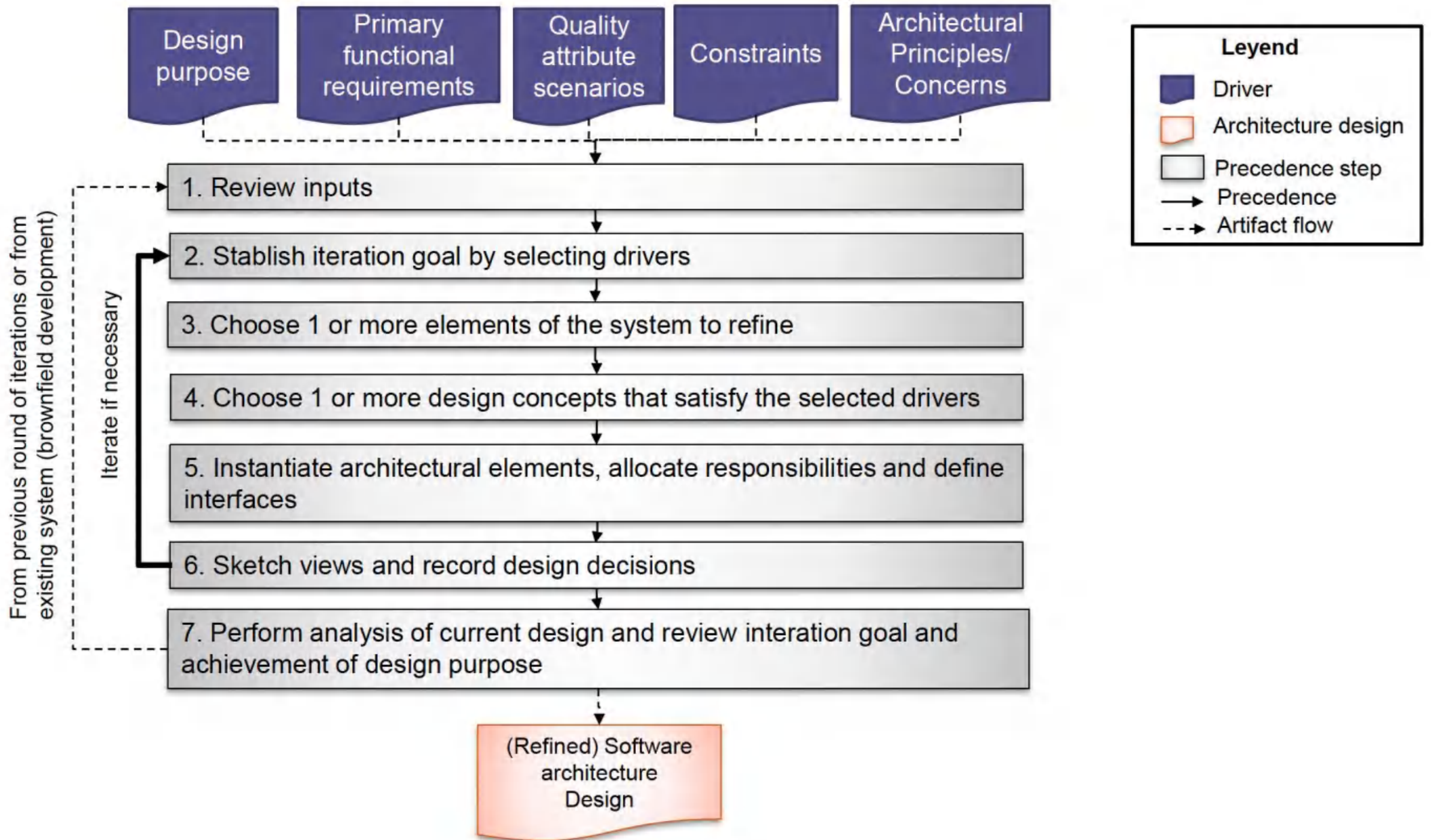


再次增加一个server时，只需要加载S4的数据，而原来的其它redis服务器都不会受到任何影响。

Figure 5-8

迭代1-产出







04 架构评估方法

架构评估方法

- 架构权衡分析方法 (ATAM)
- 成本效益分析方法 (CBA)

ATAM (架构权衡分析方法)

- 重点
 - 识别、分析和评估架构决策对系统质量属性的影响
 - 确保关键的质量属性在早期设计中得到满足
- ATAM 的简化版本:
 - 展示业务驱动因素
 - 展示架构
 - 识别架构方法
 - 生成质量属性效用树
 - 分析架构方法
 - 展示结果

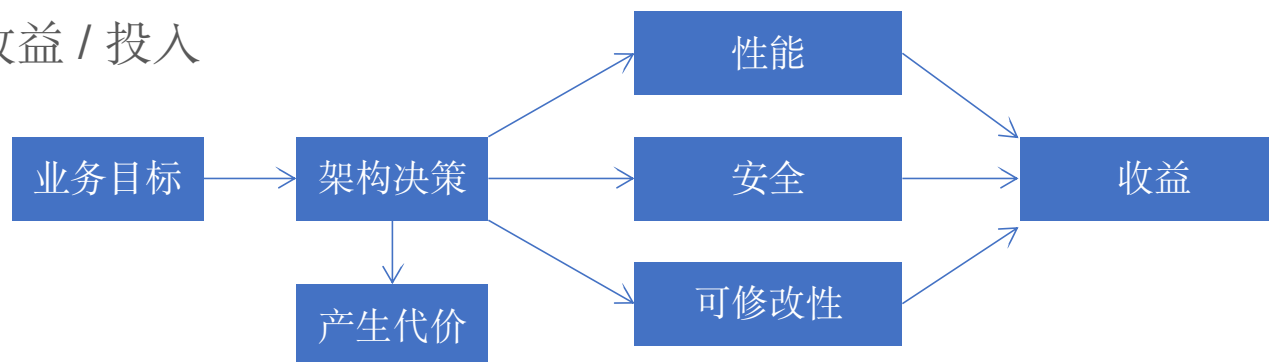
成本效益分析方法 (CBAM)

1. 重点:

1. 评估不同方案（包括架构决策）的成本和收益;
2. 旨在做出最经济合理的决策;

2. 过程

1. 选择场景和架构策略
2. 评估质量属性收益
3. 量化每种策略的收益
4. 量化成本和进度影响
5. 计算每种方案的期望值
 1. $VFC \text{ (value for cost)} = \text{收益} / \text{投入}$
6. 制定架构设计决策



架构决策记录 (ADR)

基本结构:

- 标题: 简短的描述性标题
- 状态: 提议、已接受、已取代
- 背景:
 - 是什么促使做出这个决定
 - 包括备选方案
- 决定: 决定以及相应的理由
- 后果: 该决定的预期影响

在 <用例/用户故事> 的背景下,
面对 <关注点>,
我们决定选择 <选项>
并忽略 <其他选项>,
以实现 <系统质量/期望结果>,
同时接受 <缺点/不期望的后果>,
因为 <附加理由>。

<https://adr.github.io/adr-templates/>



05 课后作业

课后作业 (1/2)

背景:

你是一个来自农村的孩子。今年春节回家，你看到很多孩子沉迷于手机游戏，生活非常单调。你把家里的一些旧书翻出来，给孩子们看，发现他们居然很喜欢。你发现并不是因为他们喜欢手机游戏，而是因为他们的生活太单调。于是你萌生了为家乡孩子们办一个图书馆的想法，丰富他们的精神世界。你从朋友那里收集到了几百本书，每本书都有一个唯一的二维码。为了方便管理，你需要设计一个图书管理系统。

基本需求:

- **借书:** 用户（借书人）使用手机扫描书籍上的二维码，并在手机上填写借书人姓名，提交借书请求。
- **还书:** 用户（借书人）在手机上提交还书请求，系统记录还书信息。
- **管理:** 管理员可以通过管理页面查看图书借阅情况，包括逾期未还的书籍、借阅频率高的书籍等，以便进行管理和调整。

扩展功能（鼓励发挥想象力）:

除了以上基本功能，你可以考虑以下扩展功能，使你的图书馆系统更加完善:

- **图书搜索:** 用户可以通过书名、作者等关键词搜索书籍。
- **图书推荐:** 系统根据用户的借阅历史推荐书籍。
- **活动发布:** 发布图书馆举办的活动信息，如读书会、讲座等。
- **用户管理:** 管理员可以管理用户信息，如添加、删除用户等。
- **捐赠管理:** 记录捐赠书籍的信息，并对捐赠者表示感谢。
- **积分系统:** 根据借阅和参与活动情况，给予用户积分奖励。

课后作业(2/2)

1.系统功能描述:

- 详细描述你的图书管理系统的主要功能，包括基本功能和扩展功能（至少选择3个扩展功能）。
- 说明每个功能解决的具体问题，以及带来的价值。

2.模块图:

- 画出你的系统的模块图。
- 清晰地标明每个模块的名称。
- 简要描述每个模块的功能和职责。
- 用箭头表示模块之间的依赖关系或调用关系。

3.组件连接器图 (选择2-3个最重要的功能):

- 选择2-3个你认为最重要的功能（例如：借书，还书，图书搜索）。
- 针对每个功能，画出组件连接器图。
- 清晰地标明每个组件的名称。
- 描述每个组件的功能。
- 用箭头表示组件之间的数据流动和调用关系。
- 说明组件之间传递的数据内容。

1.部署图:

- 画出你的系统的部署图。
- 清晰地标明每个节点的名称（例如：服务器、手机、数据库等）。
- 说明每个节点上部署的组件或模块。
- 用箭头表示节点之间的连接方式（例如：网络连接）。
- 考虑系统的可扩展性和高可用性。

提示:

- 这是一个开放性的作业，鼓励大家发挥想象力，设计出具有创新性的图书管理系统。
- 在设计过程中，可以参考现有的图书管理系统，但不要完全照搬。
- 要充分考虑系统的实用性和易用性，以及未来的可扩展性。
- 有问题可以随时提问，我会尽力解答。

THANK YOU

北京交通大学软件学院

